



Universidad Nacional Autónoma de México

Facultad de Ingeniería

SISTEMAS OPERATIVOS

Grupo: 07

Profesor:

DR. GUNNAR EYAL WOLF ISZAEVICH

Sistemas operativos inmutables

Alumno:

Corona Lopez José Francisco

Escobar Hernández Angel Emiliano

Fecha de entrega: 05-10-2026

Índice

Introducción.....	3
Marco teórico	3
¿Cómo se organizan los archivos en Unix?	3
¿Qué es usrmerge?	4
¿Qué es una actualización atómica?	4
¿Qué es un snapshot y qué es un rollback?.....	5
Desarrollo	5
Android	5
Fedora Silverblue.....	6
openSUSE MicroOS	7
NixOS	7
Comparación de los mecanismos.....	8
Límites: los datos y las aplicaciones.....	8
Conclusiones.....	9
Referencias	9

Introducción

En esta investigación se busca explicar cómo funcionan los sistemas operativos inmutables, en particular cómo se actualizan y cómo se pueden recuperar cuando una actualización falla. Escogimos el tema porque actualizar el sistema es algo que hacemos seguido y casi nunca nos preguntamos qué pasa si el proceso se corta a la mitad o si la versión nueva no funciona como esperábamos.

Cuando le propusimos el tema al profesor nos aclaró que formalmente un sistema inmutable no es un sistema operativo diferente, sino una forma de configurar la instalación, principalmente desde espacio de usuario, por lo que en este trabajo hablamos de cómo se organiza, se distribuye y se actualiza la instalación y no de un kernel nuevo.

Primero explicamos cómo se acomodan los archivos en Unix y el cambio conocido como usrmerge, después los conceptos de actualización atómica, snapshot y rollback, y posteriormente revisamos cuatro casos: Android, que es el ejemplo más conocido, Fedora Silverblue, openSUSE MicroOS y NixOS. Al final comparamos qué recupera cada uno y qué límites tienen, sobre todo con los datos que cambian fuera de la base. Los recorridos que describimos se basan en la documentación de cada sistema y no son pruebas que hayamos ejecutado nosotros.

Marco teórico

¿Cómo se organizan los archivos en Unix?

En Unix los archivos forman un árbol que empieza en la raíz, que se escribe /. No hay que confundirla con /root, que normalmente es el directorio personal de la cuenta de administrador. La organización habitual en Linux la describe el Filesystem Hierarchy Standard (FHS), que sirve como referencia aunque no todas las distribuciones lo siguen al pie de la letra, y Android tiene su propio acomodo (Linux Foundation, 2015). Las rutas que más nos importan para este tema son las siguientes:

Ruta	Uso habitual	Aclaración
/usr	Programas instalados, bibliotecas y recursos compartidos.	Incluye /usr/bin y /usr/lib. No es la carpeta del usuario.
/etc	Configuración del equipo y de los servicios.	En Silverblue se combina en cada despliegue.
/var	Información que cambia mientras el sistema trabaja.	Registros, cachés y datos de servicios.
/home	Archivos personales de los usuarios.	En Silverblue apunta a /var/home.
/boot	Archivos que se usan para arrancar.	Kernel y otros componentes, según la instalación.

Ruta	Uso habitual	Aclaración
/bin, /sbin y /lib	Antes, programas y bibliotecas esenciales.	Con usrmerge son enlaces a /usr.

Tabla 1. Rutas principales del árbol de directorios.

Que una carpeta aparezca en el árbol no quiere decir que tenga su propia partición, pues el directorio es una ubicación y la partición es una división del disco. Montar un sistema de archivos es hacer que su contenido aparezca en una ruta del árbol, por ejemplo /home puede estar en el mismo sistema de archivos que la raíz o puede ser el punto de montaje de otro distinto. Esto es importante porque se puede tener una base de solo lectura y dejar los datos en otra parte donde sí se escribe.

También hay que distinguir entre permisos y montaje. No tener permiso de escritura sobre un archivo es una restricción del usuario, mientras que un montaje de solo lectura impide escribir en todo ese sistema de archivos, por lo que para decir que una ruta es inmutable no basta con verla, hay que revisar cómo está montada y administrada en el sistema concreto.

¿Qué es usrmerge?

usrmerge junta dentro de /usr los programas y bibliotecas que antes estaban repartidos entre la raíz y /usr. Las rutas viejas se conservan como enlaces simbólicos: /bin apunta a /usr/bin, /sbin a /usr/sbin y /lib a /usr/lib, así que un programa que busca /bin/bash sigue llegando a /usr/bin/bash (Debian, s. f.-b).

Parece suficiente con crear los enlaces, pero hay paquetes y herramientas que guardan rutas o esperan los archivos en cierto lugar, y al migrar una instalación que ya existía eso se tiene que revisar. Las notas de publicación de Debian 12 ya piden el acomodo merged-/usr y describen problemas para identificar archivos a través de rutas equivalentes, y el Comité Técnico de Debian tuvo que tomar decisiones formales sobre el tema, como la de 2019 sobre el comportamiento predeterminado de debootstrap (Debian, s. f.-a, s. f.-b). El profesor nos comentó que la adopción en Debian fue de 2018 a 2022 y que costó bastante llegar a un acuerdo, pues lo difícil era no romper las instalaciones existentes.

usrmerge facilita concentrar los componentes del sistema en /usr, pero por sí solo no lo vuelve de solo lectura ni agrega versiones para regresar ni actualizaciones atómicas. Para eso hace falta un mecanismo de actualización, que es lo que vemos en los siguientes apartados.

¿Qué es una actualización atómica?

En una actualización normal, por ejemplo con apt o dnf, los paquetes se reemplazan con el sistema funcionando, y si el proceso se corta a la mitad puede quedar una mezcla de archivos nuevos y viejos. En un sistema inmutable la base no se modifica

libremente mientras se usa, para actualizarla se prepara una versión nueva aparte mediante el mecanismo del sistema, mientras que la configuración y los datos se pueden seguir modificando según el diseño de cada instalación.

Una actualización atómica tiene un momento definido en el que se elige la versión nueva ya preparada, por lo que o se queda la versión anterior o se activa la nueva completa. Hay que aclarar el alcance de la palabra atómica, ya que cambiar la versión que se va a arrancar no cambia al mismo tiempo todos los datos y procesos, y que la versión nueva esté completa no garantiza que funcione bien, pues puede traer un error en un controlador o en una aplicación.

Aquí nos sirvió el capítulo 42 de *Operating Systems: Three Easy Pieces*, donde Arpaci-Dusseau y Arpaci-Dusseau (2023) explican por qué un corte de energía entre dos escrituras puede dejar estructuras incompletas en el disco y cómo lo resuelven técnicas como fsck y el journaling. Esa protección mantiene consistente al sistema de archivos, pero no dice nada sobre si la combinación de programas instalada es la correcta, es por ello que distinguimos tres cosas: la consistencia del almacenamiento, la selección de una versión del sistema y el funcionamiento de las aplicaciones. Los mecanismos de actualización que revisamos trabajan en el segundo nivel.

¿Qué es un snapshot y qué es un rollback?

Un snapshot guarda una vista del estado de un sistema de archivos o de una parte de él. En Btrfs se puede sacar un snapshot de un subvolumen que comparte los bloques que ya existen y usa copia en escritura cuando hay cambios, es decir, el bloque modificado se escribe aparte y la vista anterior sigue apuntando al original. Por eso no equivale a copiar todo el disco, y tampoco incluye automáticamente los subvolúmenes anidados (Btrfs, s. f.).

Rollback es regresar a una versión o estado anterior que todavía está guardado. Un snapshot puede servir para hacerlo, pero hay mecanismos de recuperación que no usan Btrfs, como veremos con Android y NixOS. Además, como el snapshot está en el mismo disco, si el disco se daña se pierden la vista actual y la guardada, por lo que no sustituye a un respaldo independiente.

Desarrollo

Android

El profesor nos hizo ver que Android es un ejemplo de instalación inmutable mucho más conocido que las distribuciones de Linux. Como no tiene que mantener tanta compatibilidad hacia atrás, le fue más fácil adoptar este tipo de actualizaciones, mientras que en Fedora, SUSE o Debian gran parte de la discusión es para no romper las instalaciones que ya existen, como pasó con usrmerge.

Android separa los componentes del sistema, los del fabricante y los datos que cambian. Entre sus particiones están system, vendor y userdata, y esta última guarda las aplicaciones que instala el usuario y sus datos, así que tener una base protegida no significa que todo el teléfono se quede sin cambios (Android Open Source Project, 2026b). Para explicar el recorrido usamos el modelo A/B clásico, aunque no todos los teléfonos actuales duplican así todas sus particiones (Android Open Source Project, 2026a).

Paso	Qué sucede
1. Preparar	El equipo funciona con el slot A y la actualización se escribe en B, que es el slot inactivo.
2. B listo	Cuando termina de escribirse y se verifica, B queda listo para arrancar.
3. Elegir B	B se elige para el siguiente arranque, pero todavía no se ha comprobado que arranque bien.
4. Probar	El equipo reinicia e intenta arrancar B.
5. Confirmar	Si pasa las comprobaciones, B se marca como exitoso.
Regreso	Si B no se confirma y se acaban los intentos, el cargador puede volver al slot válido anterior.

Tabla 2. Recorrido de una actualización A/B clásica.

Con este recorrido se distingue una falla al preparar la actualización de una falla al arrancarla. El regreso depende de que el slot anterior siga siendo válido y de las políticas del dispositivo, y cambiar de slot no devuelve userdata a un estado anterior (Android Open Source Project, 2026a, 2026b).

Para no duplicar todas las particiones existe Virtual A/B, que durante la actualización usa snapshots con copia en escritura sobre las particiones dinámicas. Después de un arranque exitoso los cambios se integran en la base, a lo que se le llama merge, y una vez hecho ya no se puede contar con regresar a la versión anterior. Estos snapshots son propios de Android y no tienen relación con los de Btrfs (Android Open Source Project, 2026c).

Verified Boot comprueba la integridad y autenticidad de los componentes del arranque, y su protección anti-rollback impide arrancar versiones viejas que la política de seguridad ya no permite. Esto tiene un objetivo distinto al rollback de recuperación, pues más que ayudar a regresar limita a qué versiones se puede regresar, y verificar de dónde viene una imagen tampoco demuestra que esté libre de errores (Android Open Source Project, 2026d).

Fedora Silverblue

Silverblue maneja su base con OSTree y rpm-ostree. En lugar de reemplazar los archivos del sistema activo, la actualización prepara otro despliegue que se usa al

reiniciar, y como los cambios se transfieren de forma incremental no hace falta descargar una copia completa del disco cada vez (OSTree, s. f.; rpm-ostree, s. f.).

El recorrido que da la documentación es consultar el estado con `rpm-ostree status`, preparar la actualización con `rpm-ostree upgrade` y reiniciar para entrar al despliegue nuevo. Si este presenta un problema, se puede elegir en el menú de arranque la entrada anterior o preparar el regreso con `rpm-ostree rollback` y reiniciar (Fedora Project, s. f.-b; rpm-ostree, s. f.).

En esta organización `/usr` es de solo lectura, `/var` guarda datos que comparten todos los despliegues y `/home` se ubica en `/var/home`. `/etc` funciona distinto, ya que cada despliegue tiene el suyo y la configuración se combina al preparar el nuevo, por lo que no hay que tratarlo igual que `/var` (Fedora Project, s. f.-a; OSTree, s. f.). Por ejemplo, si la base nueva falla con el controlador de video, regresar al despliegue anterior recupera la base, pero lo que cambió en `/var` se queda como está. Las aplicaciones de Flatpak se manejan aparte, mientras que los paquetes RPM en capas sí forman parte del despliegue (Fedora Project, s. f.-a).

openSUSE MicroOS

En MicroOS se usa `transactional-update`. Primero saca un snapshot de la raíz, después aplica los cambios de paquetes sobre esa vista nueva sin tocar la raíz que está en uso, y al terminar deja el snapshot nuevo de solo lectura y lo selecciona como raíz para el siguiente arranque. Esa selección es el punto de cambio atómico, y si algo falla durante la preparación el intento se descarta (Kukuk y Forster, 2021).

Aquí sí se usan snapshots de Btrfs, por lo que los archivos que no cambiaron comparten bloques con la vista anterior y lo modificado se guarda con copia en escritura, así la raíz actual y la nueva pueden convivir sin duplicar todo su contenido (Btrfs, s. f.).

Para regresar a mano se usa `transactional-update rollback` seguido del número de un snapshot que todavía exista y después se reinicia. Es importante indicar el número, pues sin argumento la herramienta toma como destino la raíz que está arrancada en ese momento, que no necesariamente es la anterior (openSUSE, s. f.). Los datos que están aparte, como `/var`, no se recuperan con el snapshot de la raíz, y el regreso automático depende de las comprobaciones de salud que estén configuradas.

NixOS

NixOS parte de una configuración declarativa en la que se escribe qué paquetes y servicios se quieren, y el sistema prepara los componentes necesarios. Las versiones conviven en rutas distintas dentro de `/nix/store`, así que una versión nueva no tiene que sobrescribir los archivos de la anterior.

Esta idea viene de dos artículos. En el primero, Dolstra et al. (2004) explican cómo guardar cada componente junto con sus dependencias en su propia ruta para que varias versiones puedan coexistir sin pisarse, lo que permite cambiar de perfil sin romper lo que ya estaba instalado. En el segundo, Dolstra et al. (2010) llevan ese diseño a una distribución completa y también reconocen sus límites, por ejemplo que al cambiar de configuración los servicios que están corriendo no cambian todos en el mismo instante y que el estado mutable, como las bases de datos, queda fuera del modelo.

Cada configuración que se construye es una generación, y si la anterior sigue guardada se puede regresar a ella con `nixos-rebuild switch --rollback` o desde el menú de arranque cuando el sistema no logra iniciar (NixOS, s. f.). En este caso no se usan snapshots de Btrfs ni todo el disco es de solo lectura, lo que se conserva son componentes y configuraciones con versión, y siguiendo a Dolstra et al. (2010) no consideramos que switch garantice atomicidad para todo el sistema.

Comparación de los mecanismos

Al poner los cuatro casos juntos se ve que inmutable no es un solo mecanismo, pues cada sistema prepara y recupera una unidad distinta:

Sistema	Qué se prepara y se elige	Qué queda fuera
Android	Un slot A/B, y en Virtual A/B snapshots que luego se integran con merge.	userdata.
Silverblue	Un despliegue de OSTree administrado por rpm-ostree.	/var, que es compartido.
MicroOS	Un snapshot Btrfs de la raíz administrado por transactional-update.	Datos separados, como /var.
NixOS	Una generación con sus componentes en /nix/store.	Estado mutable de los servicios.

Tabla 3. Qué recupera cada mecanismo.

En todos hace falta que la versión anterior siga guardada, y en ninguno se recuperan solos los datos del usuario ni el estado de los servicios. Además, puede hacer falta reiniciar, y la política anti-rollback puede impedir usar una versión vieja aunque siga en el equipo.

Límites: los datos y las aplicaciones

Para ver un límite concreto usamos un ejemplo hipotético. Una aplicación guarda información en una base de datos, actualizamos el sistema y la versión nueva cambia el formato de esos datos. Después encontramos un problema y regresamos la base, pero ahora la aplicación vieja se encuentra con datos escritos por la nueva, y si no entiende el formato el servicio sigue fallando aunque el rollback del sistema se haya

hecho bien. Por eso además del rollback hay que planear las migraciones y tener respaldos, que es justo el límite del estado mutable que discuten Dolstra et al. (2010).

Separar las aplicaciones de la base también tiene consecuencias. Brockmeier (2024) describe en LWN la propuesta de Aeon, un escritorio de openSUSE con una base reducida y las aplicaciones aparte, lo que muestra que la inmutabilidad también cambia la forma de instalar software. Por su parte, Holwerda (2024) cuenta en OSNews que una aplicación de Flatpak no veía los diccionarios de idioma que sí estaban instalados en el sistema. Tomamos ambos como casos publicados en 2024 y no como algo que pase con todas las versiones de openSUSE o con todas las aplicaciones de Flatpak.

Conclusiones

Con esta investigación nos quedamos con que preparar otra base sirve para tener una versión conocida a la cual regresar si una actualización falla, y los cuatro sistemas que revisamos lo hacen, pero cada uno guarda algo distinto y lo elige de forma distinta, con slots en Android, despliegues en Silverblue, snapshots en MicroOS y generaciones en NixOS. Lo que ninguno resuelve solo son los datos, pues regresar la base no los regresa, por lo que antes de hacer un rollback hay que revisar qué quedó fuera y si el programa anterior todavía entiende lo que cambió la versión nueva.

A nuestro parecer el tema junta varias cosas que en la materia vimos por separado, como los sistemas de archivos con el montaje y la copia en escritura, el arranque con la selección de una base y la seguridad con la verificación de imágenes. Al comparar los sistemas nos sirvió más preguntarnos qué preparan y qué recuperan que quedarnos con la etiqueta de inmutable, y la pregunta que nos parece más útil antes de actualizar es a qué versión podemos volver si algo sale mal y qué información habrá cambiado fuera de ella.

Referencias

Android Open Source Project. (2026a, 17 de junio). *A/B (seamless) system updates*.
<https://source.android.com/docs/core/ota/ab>

Android Open Source Project. (2026b, 17 de junio). *Partitions overview*.
<https://source.android.com/docs/core/architecture/partitions>

Android Open Source Project. (2026c, 13 de julio). *Virtual A/B overview*.
https://source.android.com/docs/core/ota/virtual_ab

Android Open Source Project. (2026d, 30 de septiembre). *Verified Boot*.
<https://source.android.com/docs/security/features/verifiedboot>

- Arpaci-Dusseau, R. H., y Arpaci-Dusseau, A. C. (2023). *Operating systems: Three easy pieces* (versión 1.10). Arpaci-Dusseau Books.
<https://pages.cs.wisc.edu/~remzi/OSTEP/>
- Brockmeier, J. (2024, 14 de junio). Aeon: openSUSE for lazy developers. *LWN.net*.
<https://lwn.net/Articles/977987/>
- Btrfs. (s. f.). *Subvolumes*. Recuperado el 4 de octubre de 2026, de
<https://btrfs.readthedocs.io/en/latest/Subvolumes.html>
- Debian. (s. f.-a). *Debian Technical Committee*. Recuperado el 4 de octubre de 2026, de <https://www.debian.org/devel/tech-ctte>
- Debian. (s. f.-b). *Release notes for Debian 12 (bookworm), 64-bit PC*.
<https://www.debian.org/releases/bookworm/amd64/release-notes/>
- Dolstra, E., de Jonge, M., y Visser, E. (2004). Nix: A safe and policy-free system for software deployment. En *Proceedings of the 18th Large Installation System Administration Conference* (pp. 79–92). USENIX Association.
https://static.usenix.org/events/lisa04/tech/full_papers/dolstra/dolstra_html/index.html
- Dolstra, E., Löh, A., y Pierron, N. (2010). NixOS: A purely functional Linux distribution. *Journal of Functional Programming*, 20(5–6), 577–615.
<https://doi.org/10.1017/S0956796810000195>
- Fedora Project. (s. f.-a). *Technical information* [Documentación archivada de Silverblue]. GitHub. <https://github.com/fedora-silverblue/silverblue-docs/blob/master/modules/ROOT/pages/technical-information.adoc>
- Fedora Project. (s. f.-b). *Updates, upgrades & rollbacks* [Documentación archivada de Silverblue]. GitHub. <https://github.com/fedora-silverblue/silverblue-docs/blob/master/modules/ROOT/pages/updates-upgrades-rollbacks.adoc>
- Holwerda, T. (2024, 28 de junio). If your immutable Linux desktop uses Flatpak, I'm going to have a bad time. *OSNews*. <https://www.osnews.com/story/140090/if-your-immutable-linux-desktop-uses-flatpak-im-going-to-have-a-bad-time/>
- Kukuk, T., y Forster, I. (2021, 28 de septiembre). *The transactional update guide* (versión 0.4). openSUSE.
<https://kubic.opensuse.org/documentation/transactional-update-guide/transactional-update.html>
- Linux Foundation. (2015). *Filesystem hierarchy standard* (versión 3.0).
https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html
- NixOS. (s. f.). *NixOS manual* (sección «Rolling back»). Recuperado el 4 de octubre de 2026, de <https://nixos.org/manual/nixos/stable/#sec-rollback>

openSUSE. (s. f.). *transactional-update(8)* [Página de manual]. Recuperado el 4 de octubre de 2026, de <https://kubic.opensuse.org/documentation/manuals/transactional-update.8.html>

OSTree. (s. f.). *Deployments*. Recuperado el 4 de octubre de 2026, de <https://ostreedev.github.io/ostree/deployment/>

rpm-ostree. (s. f.). *Client administration*. Recuperado el 4 de octubre de 2026, de <https://coreos.github.io/rpm-ostree/administrator-handbook/>